



Java et la Programmation Orientée Objet



Houssem MAHMOUDI
houssem.mahmoudi@ensicaen.fr

Ensicaen©2026 - HM



Chapitre 8 : Les collections

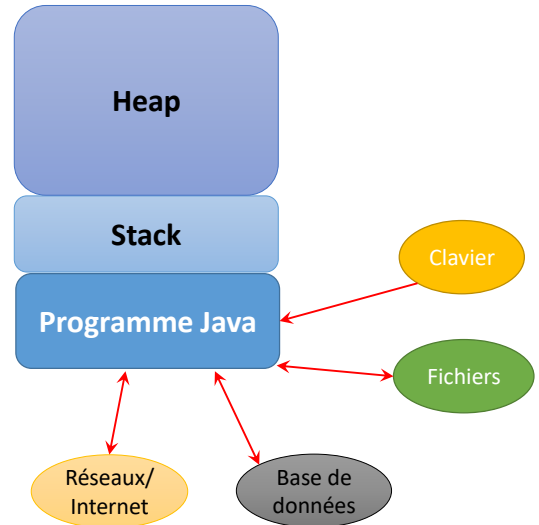
1. Introduction aux collections
2. Interfaces de Collection
3. Classes de l'interface List
4. Classes de l'interface Set
5. Classes de l'interface Map

Ensicaen©2025 - HM

2

I. Introduction aux collections

- Un programme Java peut accepter des données de plusieurs sources.
- Java utilise la notion de collection pour stocker, manipuler et transmettre des données.
- Les collections sont des objets qui groupent plusieurs éléments dans une seule unité (structure).
- Elles offrent des méthodes pour ajouter, supprimer, et accéder aux éléments.



II. Interfaces de Collections

- **Collection** : c'est l'interface de base de toutes les collections. Elle appartient au package java.util et définit les opérations de base, comme add(), remove(), size(), etc.
- **List** : une interface ordonnée qui accepte les doublons. Les éléments sont accessibles par leur index.
- **Set** : une collection (interface) qui n'accepte pas les doublons. Il n'y a pas d'ordre spécifique.
- **Map** : une interface qui stocke des paires clé-valeur. Les clés sont uniques.

III. Classes de l'interface List

- Deux classes qui implémentent l'interface List : **ArrayList** et **LinkedList**.

1. ArrayList : Une liste basée sur un tableau dynamique, rapide en accès mais lente en insertion/suppression au milieu.

```
import java.util.ArrayList;
public class TestCollection {
    public static void main(String[] args) {
        ArrayList<String> liste = new ArrayList<>();
        liste.add("Sonia");
        liste.add("Sophie");
        liste.add("Charlie");
        liste.add("Joan");
        System.out.println(liste.get(2));
        liste.remove("Sophie");
        System.out.println(liste);
    }
}
```

III. Classes de l'interface List

La Javadoc nous fournit la liste complète des méthodes de la classe **ArrayList**, ainsi que ses constructeurs.

Method Summary

All Methods	Instance Methods	Concrete Methods
Modifier and Type	Method	Description
void	<code>add(int index, E element)</code>	Inserts the specified element at the specified position in this list.
boolean	<code>add(E e)</code>	Appends the specified element to the end of this list.
boolean	<code>addAll(int index, Collection<? extends E> c)</code>	Inserts all of the elements in the specified collection into this list, starting at the specified position.
boolean	<code>addAll(Collection<? extends E> c)</code>	Appends all of the elements in the specified collection to the end of this list, in the order that they are returned by the specified collection's Iterator.
void	<code>clear()</code>	Removes all of the elements from this list.
Object	<code>clone()</code>	Returns a shallow copy of this ArrayList instance.
boolean	<code>contains(Object o)</code>	Returns true if this list contains the specified element.
void	<code>ensureCapacity(int minCapacity)</code>	Increases the capacity of this ArrayList instance, if necessary, to ensure that it can hold at least the number of elements specified by the minimum capacity argument.
boolean	<code>equals(Object o)</code>	Compares the specified object with this list for equality.

Constructor Summary

Constructors	Description
<code>ArrayList()</code>	Constructs an empty list with an initial capacity of ten.
<code>ArrayList(int initialCapacity)</code>	Constructs an empty list with the specified initial capacity.
<code>ArrayList(Collection<? extends E> c)</code>	Constructs a list containing the elements of the specified collection, in the order they are returned by the collection's iterator.

III. Classes de l'interface List

2. **LinkedList** : Classe qui implémente l'interface List, basée sur une liste chaînée, rapide en insertion/suppression mais plus lente en accès.

```
import java.util.LinkedList;
public class TestCollection {
    public static void main(String[] args) {
        LinkedList<Integer> list = new LinkedList();
        list.add(17);
        list.addFirst(55);
        list.addLast(60);
        list.add(55);
        System.out.println("Liste des valeurs:" + list);
    }
}
```

Liste des valeurs:[55, 17, 60, 55]

III. Classes de l'interface List

La Javadoc nous fournit la liste complète des méthodes de la classe **LinkedList**.

Method Summary

E	<code>poll()</code>	Retrieves and removes the head (first element) of this list.
E	<code>pollFirst()</code>	Retrieves and removes the first element of this list, or returns null if this list is empty.
E	<code>pollLast()</code>	Retrieves and removes the last element of this list, or returns null if this list is empty.
E	<code>pop()</code>	Pops an element from the stack represented by this list.
void	<code>push(E e)</code>	Pushes an element onto the stack represented by this list.
E	<code>remove()</code>	Retrieves and removes the head (first element) of this list.
E	<code>remove(int index)</code>	Removes the element at the specified position in this list.
boolean	<code>remove(Object o)</code>	Removes the first occurrence of the specified element from this list, if it is present.
E	<code>removeFirst()</code>	Removes and returns the first element from this list.
boolean	<code>removeFirstOccurrence(Object o)</code>	Removes the first occurrence of the specified element in this list (when traversing the list from head to tail).
E	<code>removeLast()</code>	Removes and returns the last element from this list.
boolean	<code>removeLastOccurrence(Object o)</code>	Removes the last occurrence of the specified element in this list (when traversing the list from head to tail).
E	<code>set(int index, E element)</code>	Replaces the element at the specified position in this list with the specified element.
int	<code>size()</code>	Returns the number of elements in this list.

IV. Classes de l'interface Set

L'interface Set en Java fait partie du package java.util et représente une collection d'éléments uniques, (sans doublons). Ses classes sont : **HashSet** et **TreeSet**.

1. **HashSet** : Basée sur une table de hachage pour stocker les valeurs uniques, ne garantit pas l'ordre des éléments et plus rapide que les autres implémentations.

```
import java.util.HashSet;
import java.util.Set;
public class TestSet {
    public static void main(String[] args) {
        Set<Integer> s = new HashSet();
        s.add(14);
        s.add(74);
        s.add(36);
        s.add(74); //valeur ignorée
        s.add(88);
        System.out.println(s);
    }
}
```

IV. Classes de l'interface Set

2. **TreeSet** : Basée sur un arbre binaire (arbre rouge-noir) et stocke les éléments dans un ordre trié.

```
import java.util.Set;
import java.util.TreeSet;
public class TestSet {
    public static void main(String[] args) {
        Set<Integer> ts = new TreeSet();
        ts.add(55);
        ts.add(66);
        ts.add(29);
        ts.add(44);
        ts.add(66); //valeur ignorée
        ts.add(33);
        ts.add(44); //valeur ignorée
        System.out.println(ts);
    }
}
```

[29, 33, 44, 55, 66]

V. Classes de l'interface Map

En Java l'interface Map du package java.util représente une collection de paires clé-valeur. Où chaque clé est unique et associé à une valeur. Ses classes sont : **HashMap** et **TreeMap**.

1. **HashMap** : Basée sur une table de hachage et ne garantit aucun ordre des éléments.

```
import java.util.HashMap;
import java.util.Map;
public class TestMap {
    public static void main(String[] args) {
        Map<String, Double> hm = new HashMap();
        hm.put("Benoit", 15.6);
        hm.put("Alice", 13.6);
        hm.put("Lucie", 16.4);
        hm.put("Benoit", 17.6); //écrase la valeur précédente
        hm.put("John", 14.8);
        System.out.println(hm);
    }
}
```

{Lucie=16.4, Alice=13.6, John=14.8, Benoit=17.6}

V. Classes de l'interface Map

2. **TreeMap** : Basée sur un arbre rouge-noir et trie les entrées selon l'ordre naturel des clés.

```
import java.util.Map;
import java.util.TreeMap;
public class TestMap {
    public static void main(String[] args) {
        Map<String, Double> tm = new TreeMap();
        tm.put("Benoit", 15.6);
        tm.put("Alice", 13.6);
        tm.put("Lucie", 16.4);
        tm.put("Benoit", 17.6); //écrase la valeur précédente
        tm.put("John", 14.8);
        System.out.println(tm);
    }
}
```

{Alice=13.6, Benoit=17.6, John=14.8, Lucie=16.4}